

Nome: Guilherme de Moraes China Costa

Data: 26/09/2026

Título: Projeto de Bloco: Engenharia de Softwares Escaláveis - TP4

Refatoração para Arquitetura Orientada a Eventos

Link da implementação:

https://github.com/GuiC0506/infnet/tree/master/projeto_bloco_engenharia_software_escalavel/TP4

O sistema de biblioteca foi refatorado para utilizar RabbitMQ na comunicação entre a aplicação principal e o microsserviço de notificações. Antes, o backend criava notificações por uma chamada HTTP síncrona. Agora, empréstimos e devoluções geram eventos que são enviados de forma assíncrona.

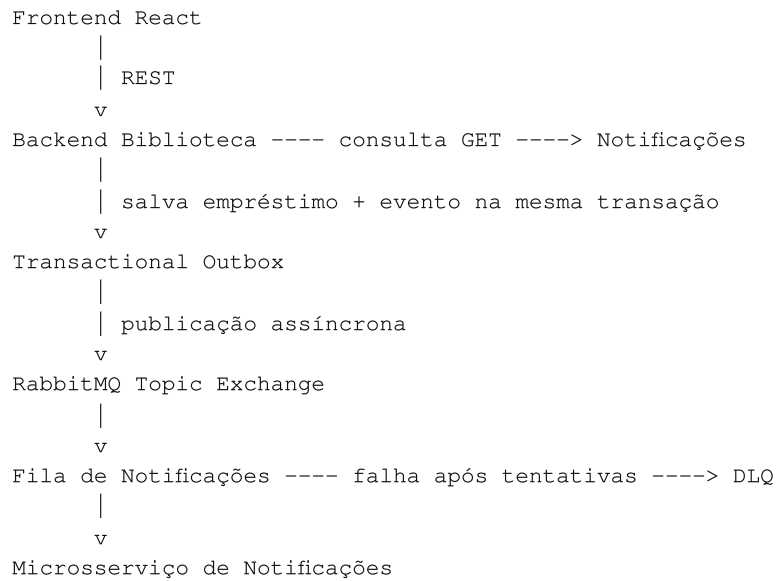
A consulta de notificações continua sendo feita por HTTP, pois é uma operação de leitura solicitada pelo frontend. A criação de notificações passou a ocorrer exclusivamente pelo consumo dos eventos.



```
guichina@Guichina: ~/infnet/projeto_bloco_engenharia_software_escalavel/TP4 [master] [!19 ?148 ] [us-east-1]
* > docker compose ps
NAME                                IMAGE                                COMMAND                                SERVICE            CREATED          STATUS
tp4-backend-1                      tp4-backend                        "java -jar app.jar"                backend            14 minutes ago  Up 14 minutes
tp4-frontend-1                    tp4-frontend                      "/docker-entrypoint..."          frontend           14 minutes ago  Up 14 minutes
tp4-notificacoes-service-1         tp4-notificacoes-service          "java -jar app.jar"                notificacoes-service 14 minutes ago  Up 14 minutes
tp4-rabbitmq-1                    rabbitmq:4.1-management-alpine    "docker-entrypoint.s..."         rabbitmq           14 minutes ago  Up 13 minutes (lthy)
4369/tcp, 5671/tcp, 0.0.0.0:5672->5672/tcp, [::]:5672->5672/tcp, 15671/tcp, 15691-15692/tcp, 25672/tcp, 0.0.0.0:15672->15672/tcp, [::]:672->15672/tcp
```

Figura 1 - Backend, frontend, RabbitMQ e serviço de notificações em execução.

Arquitetura implementada



O padrão Transactional Outbox foi usado para que o empréstimo e seu evento sejam gravados juntos. Se o RabbitMQ estiver indisponível, o evento permanece pendente no banco da Biblioteca e é publicado quando o broker volta a funcionar.

Fluxo de um empréstimo

1. O usuário registra um empréstimo.
2. O backend salva o empréstimo e o evento na tabela de Outbox.
3. O publicador envia o evento ao RabbitMQ.
4. O RabbitMQ encaminha a mensagem para a fila de notificações.
5. O microsserviço consome a mensagem e salva a notificação.
6. O frontend consulta as notificações do leitor por HTTP.

Eventos criados

- `biblioteca.emprestimo.registrado.v1`
- `biblioteca.emprestimo.devolvido.v1`

Os eventos possuem identificador único, tipo, versão, data, dados do empréstimo, leitor e livro. A versão no contrato e na routing key permite evoluir as mensagens futuramente.

Padrões de mensagens

Padrão	Uso no projeto
Publish/Subscribe	Um TopicExchange recebe os eventos e permite adicionar novos consumidores sem alterar o backend.
Fila de trabalho	A fila durável distribui mensagens entre instâncias do serviço de notificações.
Transactional Outbox	Evita a perda do evento entre o commit do banco e a publicação no RabbitMQ.
Retry e Dead Letter Queue	Uma mensagem que falha é tentada três vezes e depois enviada para a DLQ.
Consumidor idempotente	O eventId impede que uma mensagem entregue novamente gere uma notificação duplicada.
Request/Response	O OpenFeign permanece somente na consulta GET de notificações.

Abstrações do Spring Boot

Foram utilizadas as abstrações do Spring Boot e Spring AMQP para reduzir o código de infraestrutura:

- `spring-boot-starter-amqp`;
- `RabbitTemplate` para publicar mensagens;
- `@RabbitListener` para consumir mensagens;
- `TopicExchange`, `Queue` e `BindingBuilder`;
- `@Scheduled` para publicar eventos pendentes da Outbox;
- configurações de `retry`, `ACK` e conexão em `application.properties`.

Infraestrutura de mensageria

O RabbitMQ declara uma fila principal para os eventos de empréstimo e devolução e uma dead-letter queue para mensagens que continuam falhando após as tentativas configuradas.

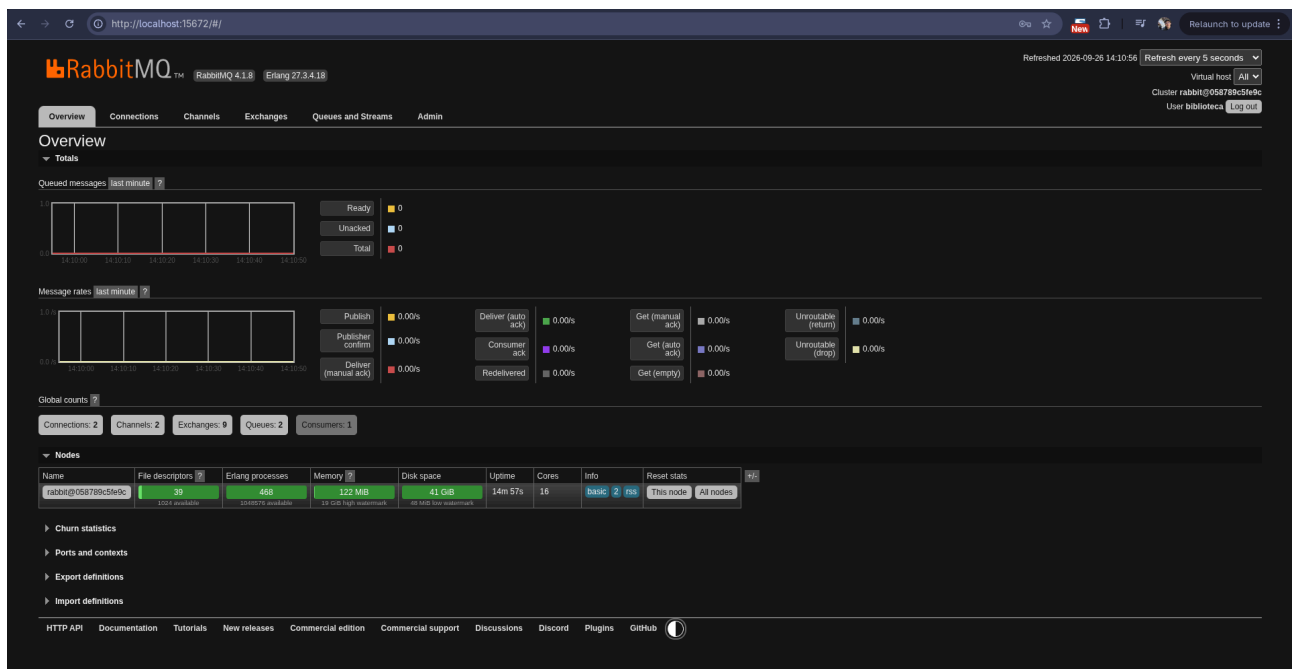


Figura 2 - Fila principal e dead-letter queue declaradas no RabbitMQ.

A fila principal possui um consumidor ativo. A DLQ permanece disponível para inspeção e tratamento manual de mensagens problemáticas.

Prós e contras da arquitetura orientada a eventos

Vantagens

- Reduz o acoplamento entre a Biblioteca e o serviço de notificações.
- Permite que o backend continue funcionando quando o consumidor está indisponível.
- Facilita a escala horizontal de consumidores.
- Permite adicionar novos consumidores aos mesmos eventos.
- Ajuda a absorver picos de requisições por meio das filas.

Desvantagens

- Existe consistência eventual: a notificação pode demorar alguns segundos.
- O RabbitMQ adiciona infraestrutura e necessidade de monitoramento.
- A depuração de fluxos distribuídos é mais complexa.
- Os contratos dos eventos precisam ser versionados.
- A entrega pode ocorrer mais de uma vez, exigindo consumidores idempotentes.

Cenários mais adequados

Essa arquitetura é vantajosa quando há vários consumidores, picos de carga, operações demoradas ou quando o produtor deve continuar funcionando mesmo que outro serviço esteja temporariamente indisponível.

Para operações simples que precisam de resposta imediata e consistência forte entre poucos componentes, uma chamada síncrona pode continuar sendo a alternativa mais simples.

Tratamento de falhas utilizado

A Outbox evita perda de eventos; o publisher confirm valida a chegada ao broker; o retry trata falhas temporárias; a DLQ separa falhas persistentes; e o eventId evita duplicações.

Demonstração da funcionalidade

O primeiro cenário demonstra o fluxo normal. Depois de registrar um empréstimo, o evento é consumido e a notificação fica disponível para o leitor. Ao registrar a devolução, uma segunda notificação é criada.



Figura 3 - Notificações geradas a partir dos dois tipos de evento.

Na imagem, o frontend apresenta as notificações geradas de forma assíncrona pelo microserviço após o consumo dos eventos publicados no RabbitMQ.

Cenário de resiliência

Também foi validado um cenário de indisponibilidade. Com o RabbitMQ parado, o empréstimo foi registrado normalmente e o evento permaneceu pendente na Outbox.

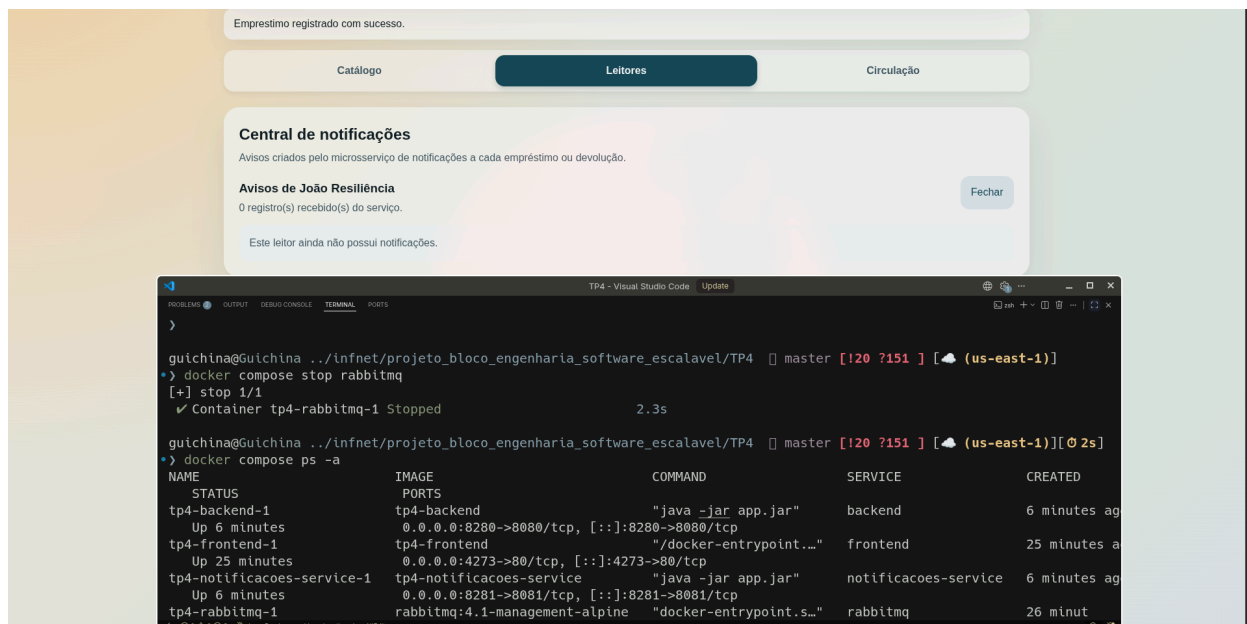


Figura 4 - Registro do empréstimo durante a indisponibilidade do RabbitMQ.

Após iniciar o broker novamente, o publicador enviou o evento pendente e a notificação apareceu automaticamente, sem repetir o empréstimo.

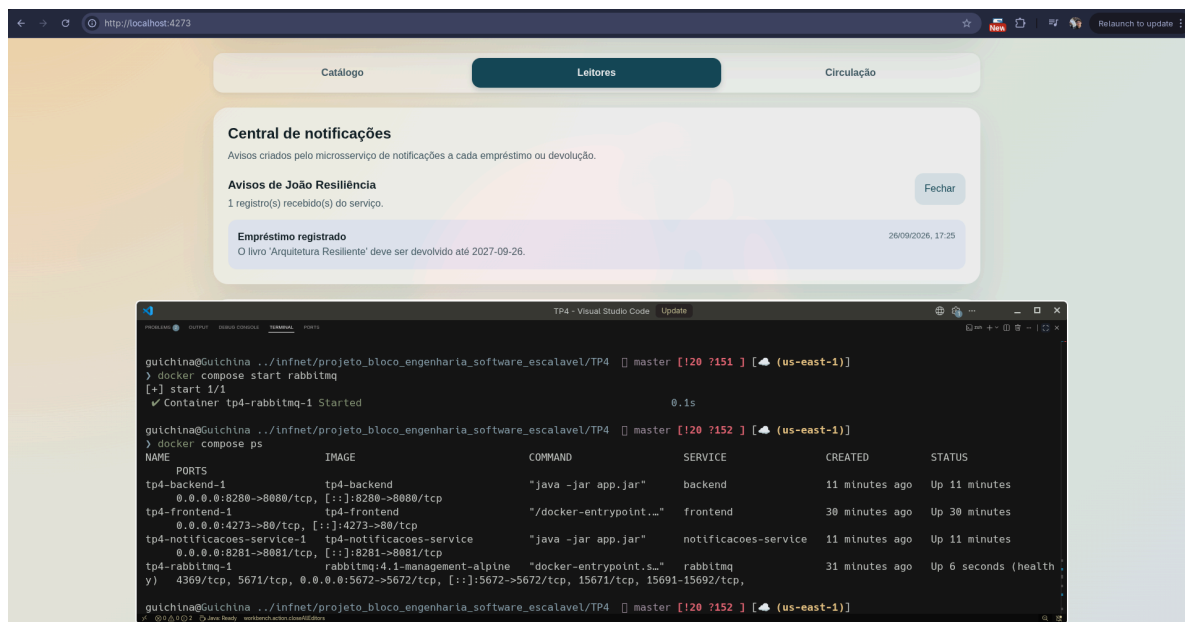
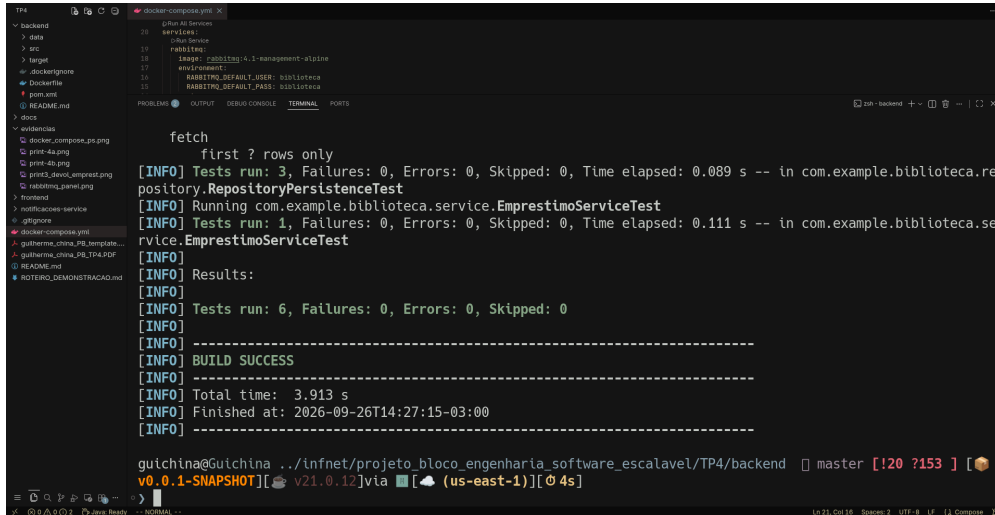


Figura 5 - Processamento do evento depois da recuperação do RabbitMQ.

Testes e conclusão

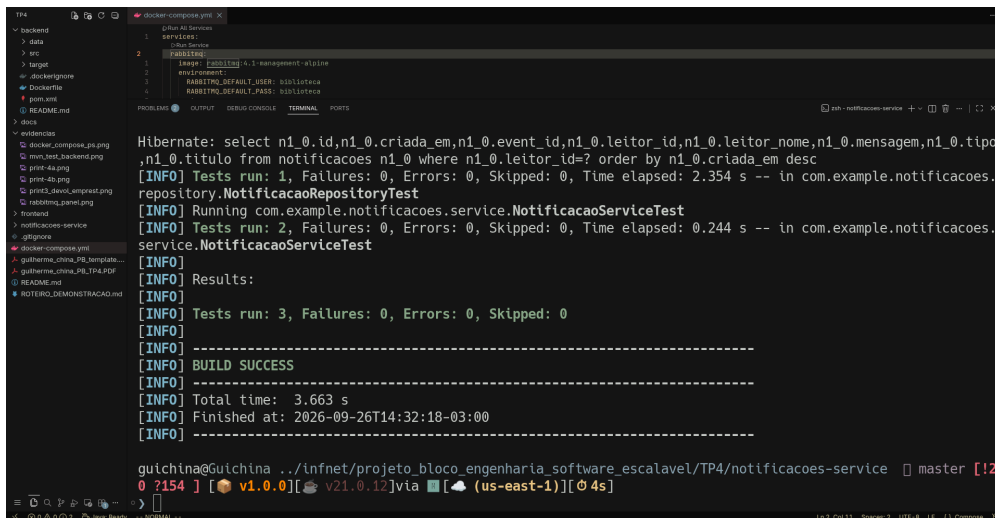
Os testes automatizados verificam as regras já existentes, o registro do evento na Outbox, o conteúdo da mensagem, a persistência da notificação recebida e a idempotência do consumidor.



```
TP4
├── backend
│   ├── data
│   ├── src
│   ├── target
│   ├── dockerignore
│   ├── Dockerfile
│   ├── pom.xml
│   └── README.md
├── docs
├── evidencias
│   ├── docker_compose_ps.png
│   ├── print-4a.png
│   ├── print-4b.png
│   ├── print_devot_emptest.png
│   └── rabbitmq_panel.png
├── frontend
├── notificacoes-service
│   ├── .gitignore
│   ├── docker-compose.yml
│   ├── guilherme_china_R8_TP4.PDF
│   ├── guilherme_china_R8_TP4.PDF
│   ├── README.md
│   └── NOTEMO_DEMONSTRACAO.md
└── TP4

fetch
first ? rows only
[INFO] Tests run: 3, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.089 s -- in com.example.biblioteca.reposit
pository.RepositoryPersistenceTest
[INFO] Running com.example.biblioteca.service.EmprestimoServiceTest
[INFO] Tests run: 1, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.111 s -- in com.example.biblioteca.se
rvice.EmprestimoServiceTest
[INFO] Results:
[INFO]
[INFO] Tests run: 6, Failures: 0, Errors: 0, Skipped: 0
[INFO]
[INFO] BUILD SUCCESS
[INFO]
[INFO] Total time: 3.913 s
[INFO] Finished at: 2026-09-26T14:27:15-03:00
[INFO]
guichina@Guichina: ~/infnet/projeto_bloco_engenharia_software_escalavel/TP4/backend master [!20 ?153] [
v0.0.1-SNAPSHOT][v21.0.12]via [us-east-1][0 4s]
```

Figura 6 - Testes automatizados do backend.



```
TP4
├── backend
│   ├── data
│   ├── src
│   ├── target
│   ├── dockerignore
│   ├── Dockerfile
│   ├── pom.xml
│   └── README.md
├── docs
├── evidencias
│   ├── docker_compose_ps.png
│   ├── mvn_test_backend.png
│   ├── print-4a.png
│   ├── print-4b.png
│   ├── print_devot_emptest.png
│   └── rabbitmq_panel.png
├── frontend
├── notificacoes-service
│   ├── .gitignore
│   ├── docker-compose.yml
│   ├── guilherme_china_R8_TP4.PDF
│   ├── guilherme_china_R8_TP4.PDF
│   ├── README.md
│   └── NOTEMO_DEMONSTRACAO.md
└── TP4

Hibernate: select n1_0.id,n1_0.criada_em,n1_0.event_id,n1_0.leitor_id,n1_0.leitor_nome,n1_0.mensagem,n1_0.tipo
,n1_0.titulo from notificacoes n1_0 where n1_0.leitor_id=? order by n1_0.criada_em desc
[INFO] Tests run: 1, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 2.354 s -- in com.example.notificacoes.
repository.NotificacaoRepositoryTest
[INFO] Running com.example.notificacoes.service.NotificacaoServiceTest
[INFO] Tests run: 2, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.244 s -- in com.example.notificacoes.
service.NotificacaoServiceTest
[INFO] Results:
[INFO]
[INFO] Tests run: 3, Failures: 0, Errors: 0, Skipped: 0
[INFO]
[INFO] BUILD SUCCESS
[INFO]
[INFO] Total time: 3.663 s
[INFO] Finished at: 2026-09-26T14:32:18-03:00
[INFO]
guichina@Guichina: ~/infnet/projeto_bloco_engenharia_software_escalavel/TP4/notificacoes-service master [!2
0 ?154] [v1.0.0][v21.0.12]via [us-east-1][0 4s]
```

Figura 7 - Testes automatizados do microserviço.

Conclusão

A refatoração removeu o acoplamento síncrono na criação de notificações e tornou o sistema mais resiliente. A solução utiliza diferentes padrões de mensagens, recursos do RabbitMQ e abstrações do Spring Boot, além de lidar com perda, duplicação e falhas persistentes.